

Manual Técnico

Arquitectura, stack y mantenimiento

A Tu Manera — v1.0.0

Manual Técnico — A Tu Manera

1. Stack y versiones (fijas, sin + ni latest)

Componente	Versión
Kotlin	1.9.24
Android Gradle Plugin	8.5.2
Gradle	8.7 (wrapper)
KSP	1.9.24-1.0.20
Compose BOM	2024.06.00
Material 3	1.2.1
Navigation Compose	2.7.7
Room	2.6.1
Coroutines	1.8.1
JDK	17
compileSdk / targetSdk	34
minSdk	24

2. Estructura de carpetas

```
app/src/main/kotlin/com/educalab/atumanera/
■■■ AtumaneraApplication.kt      # DI manual (lazy), seeding al arranque
■■■ MainActivity.kt              # host único de Compose
■■■ domain/
■   ■■■ model/                  # Enums.kt, GridModels.kt – modelos puros
■   ■■■ logic/                  # GridEngine, MetricsCalculator, BudgetManager,
■                               # MissionEvaluator, UnlockEvaluator (sin Android)
■■■ data/
■   ■■■ local/
■   ■   ■■■ entity/Entities.kt  # 16 entidades Room
■   ■   ■■■ dao/                 # CoreDataos.kt, ProgressDaos.kt
■   ■   ■■■ AppDatabase.kt
■   ■■■ repository/CityRepository.kt # orquesta domain + Room
■   ■■■ seed/                    # CatalogSeed, MissionSeed, DatabaseSeeder
■■■ ui/
■   ■■■ theme/                   # Color.kt, Theme.kt
■   ■■■ components/              # CityGridCanvas, CommonComponents, CategoryVisuals...
■   ■■■ screens/                 # 7 pantallas + BuildScreen genérica
■   ■■■ navigation/AppNavGraph.kt
■   ■■■ CityViewModel.kt / MissionsViewModel.kt / CollectionViewModel.kt
■■■ util/AppPreferences.kt       # SharedPreferences local (onboarding, sonido, háptica)
```

3. Arquitectura

MVVM + Repository, con dominio puro desacoplado de Android:

- domain/ no importa android.* ni androidx.room.*: se compila y testea con Kotlin puro.

- `data/repository/CityRepository` es el único punto que combina Room con el motor de reglas de `domain/logic`. Ningún Composable ni ViewModel ejecuta SQL directamente.
- `ui/*ViewModel` exponen `StateFlow` derivados con `combine/flatMapLatest` de los `Flow` de Room; no contienen reglas de negocio, sólo orquestan la presentación y delegan acciones al repositorio.

4. ViewModels

- `CityViewModel`: combina usuario, ciudad, casillas+catálogo y métricas en un único `StateFlow<CityBoardState>`. Expone `place(row, col, infraId)` y `remove(row, col)`, y un `SharedFlow<CityEvent>` para feedback puntual (misión completada, insignia ganada, rechazo).
- `MissionsViewModel`: une el catálogo de 30 misiones con el progreso real por usuario/ciudad.
- `CollectionViewModel`: une catálogo de insignias/decoraciones con lo desbloqueado por el usuario.

5. Repositorio — flujo de `placeInfrastructure`

- Verifica que la ciudad y la casilla existen.
- Verifica que la casilla está libre (`PlacedInfrastructureDao.getAt()`).
- Verifica presupuesto vía `BudgetManager.place(...)`.
- Inserta `PlacedInfrastructureEntity`.
- Si es una carretera, enlaza `RoadConnectionEntity` con vecinos de carretera (4 direcciones).
- Llama a `finalizeCityUpdate`, que:
- Reconstruye el `List<TileSnapshot>` de la ciudad.
- Calcula métricas con `MetricsCalculator` (usa `GridEngine` internamente).
- Persiste `ServiceCoverageEntity` por categoría de servicio.
- Inserta una fila `CityMetricEntity` (historial) y recorta a 100 entradas.
- Evalúa las 30 misiones con `MissionEvaluator` y actualiza `MissionProgressEntity`.
- Otorga XP y actualiza `ProgressEntity`.
- Evalúa insignias/decoraciones con `UnlockEvaluator` y persiste lo nuevo.

Toda la función es `suspend` y se ejecuta en el `Dispatcher` por defecto de Room/Coroutines; los `Flow` de Room hacen que la UI se actualice automáticamente sin llamadas manuales de refresco.

6. Room

- 16 entidades, todas con `@PrimaryKey(autoGenerate = true)`.
- Claves foráneas con `onDelete = ForeignKey.CASCADE` en todas las relaciones.

- Índices únicos que **son** las reglas de negocio a nivel de base de datos (una infraestructura por casilla, un progreso de misión por usuario/ciudad/misión, etc.).
- AppDatabase expone 16 DAOs; el patrón singleton (getInstance) evita múltiples instancias de la base de datos.
- exportSchema = true: preparado para migraciones versionadas en el futuro (v1.0.0 no requiere ninguna).

7. Dependencias declaradas (app/build.gradle.kts)

Ver el archivo completo en el repositorio. Resumen: Compose (ui, material3, material-icons-extended), navigation-compose, room-runtime + room-ktx + room-compiler (vía KSP), kotlinx-coroutines-android, y para tests: junit, kotlinx-coroutines-test, turbine, room-testing, robolectric, androidx.test:core, truth.

8. Permisos

Ninguno declarado en AndroidManifest.xml. Sin INTERNET, sin almacenamiento externo, sin ubicación.

9. Build

```
./gradlew clean
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
```

Ver BUILD_REPORT.md para el resultado real y honesto de estos comandos en el entorno usado para construir este proyecto, y para el detalle de qué verificaciones alternativas sí se pudieron ejecutar (compilación del dominio puro con kotlinc oficial y ejecución real de 45 tests).

10. Mantenimiento y ampliaciones

- **Añadir una infraestructura nueva:** agregar una fila en CatalogSeed.infrastructureTypes(), un icono en tools/generate_icons.py (o un drawable manual), y una entrada en infraIconRes() (CategoryVisuals.kt).
- **Añadir una misión:** agregar un MissionSeedDefinition en MissionSeed.missions() con sus RequirementSeed; el motor de evaluación ya soporta los 5 tipos de requisito sin cambios de código.
- **Añadir una insignia/decoración:** agregar la fila en CatalogSeed y una UnlockCondition en UnlockEvaluator.defaultBadgeConditions() / defaultDecorationConditions().
- **Cambiar el tamaño de la ciudad:** ajustar DEFAULT_CITY_ROWS/DEFAULT_CITY_COLS en DatabaseSeeder.kt; el GridEngine y el CityGridCanvas son genéricos en filas/columnas.